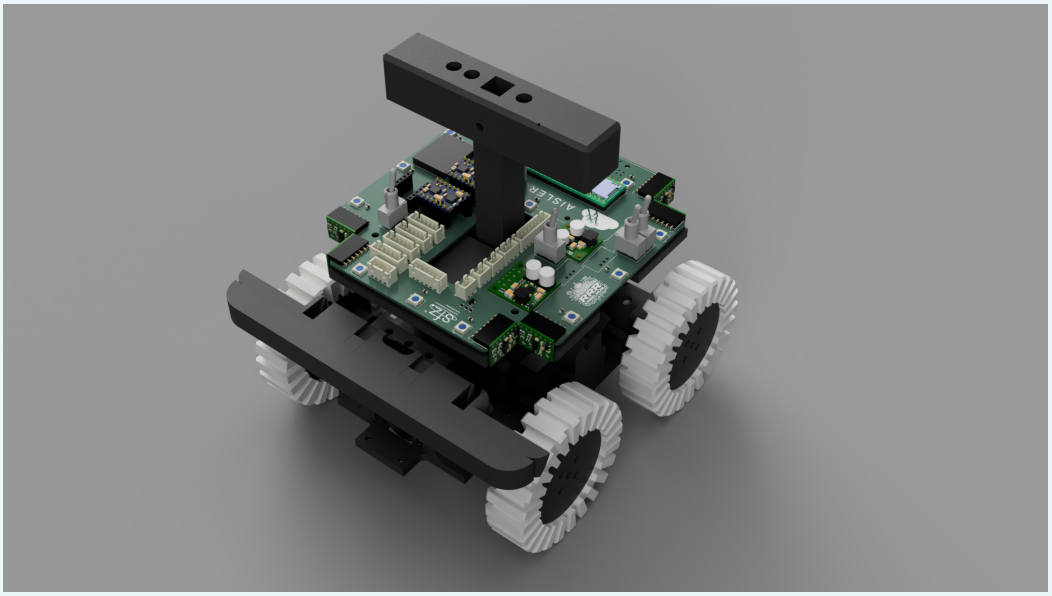
Technical Description Paper

RoboCup Junior Rescue Maze

Team Kabelmüsli

Competition season 2026

Authors. Joel Dankert, Felix Hafner, Simon Müther, David Schumacher



Contents

Abstract

We built a completely new RoboCupJunior Rescue Maze robot for the 2026 season. The goal was a compact, competition-ready platform with higher consistency and better integration than our earlier robots. Compared to previous seasons, this build replaces a cable-heavy architecture with a central custom PCB, a tighter mechanical package, and a cleaner split between embedded sensing, Raspberry Pi decision making, and telemetry.

Our robot uses four driven wheels, a passive suspension based on coupled rocking axles, custom cast silicone tires, eight side-facing VL53L1X distance sensors, three forward multi-target ToF sensors, RGB and reflectance floor sensing, dual ultra-wide-angle victim cameras, and a directional rescue-kit mechanism. The software stack combines live sensor fusion, a three-dimensional tile map, shortest-path navigation, persistent silver checkpoint recovery, dual-model victim recognition, and a local dashboard.

This paper focuses on the design rationale, the integration strategy, and the validation loops that shaped the final system. Special attention is given to compact packaging, maintainability, checkpoint recovery, and synthetic-data victim training.

Key Contributions. Central custom PCB, coupled suspension with cast silicone tires, ultra-wide-angle victim AIs, and fingerprint-based silver checkpoint recovery.

1 Introduction and Scope

In this paper, we document our current 2026 Rescue Maze robot and the engineering decisions that shaped it.

1.1 Document Goal

This paper explains how the current robot was designed, integrated, and validated for RoboCupJunior Rescue Maze. It focuses on the real competition robot and connects the task directly to subsystem design, implementation choices, and test results.

1.2 Version Scope

This paper describes only our current 2026 robot. It is not a lightly modified carry-over platform but a complete redesign with a new mechanical package, a new electronics layout, and a more disciplined overall system integration. Only parts of the software stack were carried over and further developed from earlier work. This specific build was prepared for the World Championship in South Korea.

The target of this redesign was higher full-run consistency than in previous seasons. We improved key software components before the hardware was finished and then transferred them onto the final compact robot during the integration phase.

This paper is structured around the main engineering layers of the system. Section 2 explains planning and integration. Sections 3 to 5 cover mechanics, electronics, and software. Section 6 summarizes test setups, reliability metrics, and the most important corrective actions. The conclusion then compresses the most important design outcomes into a short final assessment. Figures are placed close to the corresponding text, even when this increases the total document length, because that makes the system easier to follow.

2 Project Planning and System Engineering

The new robot was shaped by a small number of strict competition-driven requirements and by our decision to avoid another late, high-risk integration cycle.

2.1 Competition-Derived Requirements

The season goal was ambitious but concrete: the robot should be capable of solving almost complete mazes with far better consistency than in previous years. This translated into several practical requirements. The robot had to become more compact, because the previous chassis concept occupied too much volume and limited packaging freedom. It had to stay stable on ramps despite a relatively high drive concept, because earlier versions tended to pitch forward when descending. It needed cleaner electrical integration, because we wanted to replace the previous "cable spaghetti" with a maintainable central board. Finally, the software had to recover from navigation problems more gracefully, because a single mapping failure should not destroy an otherwise strong run.

Competition challenge	Requirement	Design response
Tight maze geometry and transport limits	Reduce overall footprint without losing obstacle capability	New compact chassis, stacked structure, central PCB, tighter sensor packaging
Ramps, bumpers, and uneven terrain	Keep all wheels engaged and avoid front tipping	Four-wheel drive, large custom wheels, low battery placement, coupled rocking axles
Fast and repeated field debugging	Make internal state observable during runs	Local dashboard with live map, sensor panels, camera feeds, sounds, and gallery
Mapping failures after black tiles or collisions	Recover useful state instead of fully restarting	Persistent silver checkpoints with continuous fingerprint matching and selective reset logic

2.2 Development Timeline, Roles, and Review Gates

We split work so that software progress could start before the final robot existed. Joel Dankert led software integration and debugging. Simon Müther focused on software and electronics. Felix Hafner mainly handled hardware, electronics, and assembly. David Schumacher worked on hardware and external presentation. All four of us entered the season with prior RoboCup experience, which made it realistic to work on multiple domains in parallel.

The rough order of work was stable even if day-to-day iteration stayed flexible. Software improvements began first on the previous robot, especially mapping, basic driving behavior, victim AI, and debugging tooling. In parallel, the new robot entered CAD design and PCB development. Once the mechanical structure and board layout stabilized, assembly and electrical bring-up followed. Only after the new hardware was sufficiently complete was the software transferred to the final robot, after which the focus shifted to tuning and full maze reliability. This also reflects our usual season pattern: we rebuild major parts of the robot for nearly every competition, and this version was specifically rebuilt for the World Championship in South Korea.

Phase	Milestone	Main owner(s)	Gate to continue
Early season	Software baseline improved on previous robot	Joel	Mapping, victim detection, and dashboard usable independent of new hardware
Mid season	CAD and PCB concept frozen	Felix, Simon	Compact layout defined, interfaces between sensors, compute, and mechanics fixed
Mid to late season	New robot assembled and powered	Felix, David, Simon	Chassis complete, board mounted, sensors connected, drive system functional
Final weeks	Software ported and tuned on final robot	Joel, Simon	Repeated full-maze runs possible and major failure loops identified

2.3 Integration Architecture

The robot is partitioned into five main modules: drivetrain and suspension, rescue-kit mechanism, sensing, embedded control, and high-level compute. Mechanically, the chassis carries the battery, axles, upper sensor plate, bumper bar, and dispenser. Electrically, a central PCB distributes power and routes signals. A dedicated microcontroller handles time-critical sensor and actuator interfacing, while the Raspberry Pi is responsible for mapping, navigation, camera inference, and telemetry. This split made it possible to test high-level behavior with old hardware and then migrate to the new platform with limited changes at interface boundaries.

Communication between modules follows a clear ownership model. The Raspberry Pi owns global state, mapping, victim logic, and operator feedback. The embedded layer owns low-level sensor readout and hardware-facing actuation. The PCB provides the physical integration point for power and buses. Cameras feed directly into the Pi, because victim processing is compute-heavy. Distance, color, gyro, and other auxiliary sensors enter through the embedded side and are normalized before they affect navigation decisions.

2.4 Interface Contracts and Failure Containment

The most important integration decision was to keep failures local whenever possible. If a sensor stream becomes unreliable, the robot should not silently continue with invalid assumptions. The embedded layer therefore serves as a boundary for sensor acquisition and hardware supervision. On the software side, map state is preserved through silver checkpoints, and resets are treated as recoverable events rather than total run-ending faults. We also simplified the system when a feature failed to prove its value. A good example is stepless driving: it was explored in software but later deprioritized when it reduced reliability without improving consistency enough to justify the added complexity.

This mindset also influenced the order of work. Instead of adding features until the end of the season, the we repeatedly reduced or delayed unstable ideas and concentrated on the subsystems that most directly improved complete maze performance: compact packaging, strong traction, robust mapping, victim handling, and fast debugging feedback.

3 Mechanical Design and Manufacturing

The 2026 robot is a fully new mechanical design centered on compact packaging, obstacle capability, and fast repairability.

3.1 Mechanical Architecture Overview

The robot is built around a main printed chassis plate that acts as the structural base for nearly all other subassemblies. The battery is placed in a central cutout in this plate, which keeps the heaviest component low and near the middle of the robot. Front and rear axle modules are mounted to the chassis, while an upper plate carries the main electronics and the distributed sensor set. At the front, a dedicated bumper bar integrates tactile switches, forward sensing, and lighting.

This layered structure solves several goals at once. First, it uses vertical space more efficiently than the previous robot, which was difficult to package cleanly. Second, it improves service access because individual modules can be removed without dismantling the entire robot. Third, it gives us fixed mounting geometry for electronics and sensors, which reduced ad-hoc wiring and made the final robot feel much more intentional.

The rescue-kit dispenser is placed above the central region of the chassis. This keeps the mechanism close to the robot center instead of hanging mass far to one side. Because the robot uses a relatively tall wheel and suspension concept, this mass placement was important for keeping pitch behavior manageable on ramps.

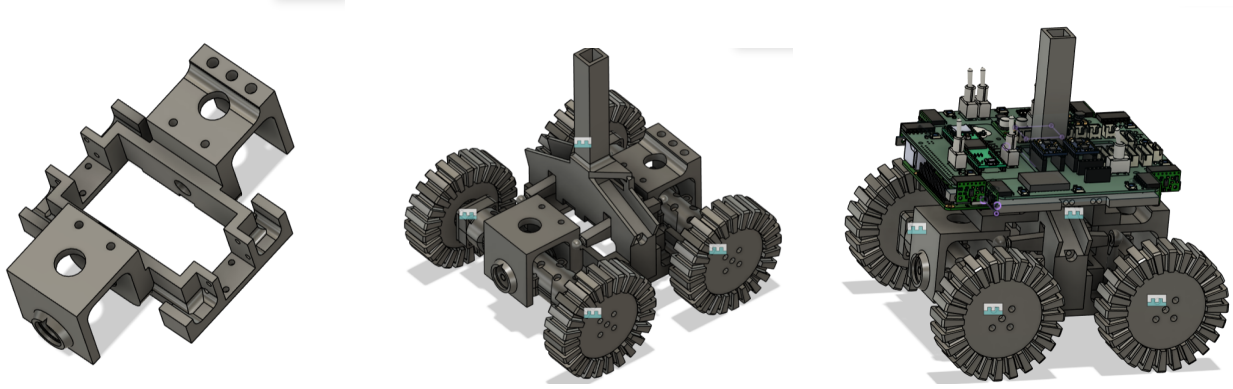


Figure 1: From left to right: the printed chassis base, the coupled axle and suspension concept, and the integrated robot assembly with upper electronics deck.

3.2 Drive, Wheel, and Suspension Subsystem

We use four directly driven wheels powered by four 12 V metal gearmotors with integrated encoders. Each wheel sits directly on a motor shaft and is fixed mechanically without an extra reduction stage. This minimizes transmission complexity and keeps the drive path stiff. The wheel itself consists of a printed PLA-CF rim, a metal motor interface, and a custom cast silicone tire. The resulting tire diameter and soft contact surface were chosen to improve grip on ramps, edges, and uneven maze elements.

Mechanically, the most unusual subsystem is the passive suspension. The front and rear axle modules can rotate relative to the chassis and are coupled through a rocker-style linkage with ball joints. When one wheel moves up over an obstacle, the mechanism redistributes the motion so that the other wheels remain in contact with the floor as much as possible. This approach is especially valuable in Rescue Maze because small losses of traction quickly propagate into heading errors, side-distance errors, and mapping mistakes.

We consider this suspension the most innovative hardware feature of the season. It allowed the robot to keep using relatively large wheels and generous ground clearance while still handling ramps, bumpers, and uneven transitions with more stability than a rigid frame. The lower-mounted battery then complements this mechanical concept by reducing the tendency to tip forward when descending.

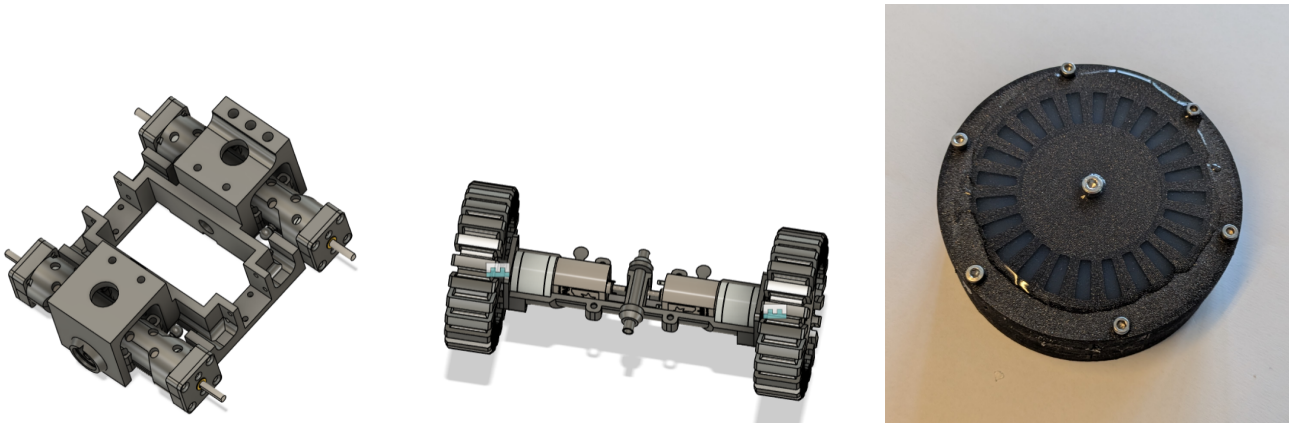


Figure 2: Drive-train details. Left: motor and shaft mounting without the upper structure. Center: axle module with wheels and central shaft linkage. Right: finished custom wheel with cast silicone tire mounted on the printed rim.

3.3 Victim Kit Deployment Mechanism

The rescue-kit mechanism stores kits in a vertical tower and ejects them through left or right ramps. A servo drives a hook that releases the staged kits into the correct path. The geometry of the hook motion determines which side receives the next kit. If the hook first moves in one direction and then in the other, the kit is guided to a specific side of the robot. This makes the mechanism directional without requiring two independent actuators.

In practice, the difficult parts were not the general principle but the tolerances. We had to control hook reset accuracy, avoid jamming, and prevent the mechanism from stopping between valid positions. These problems were first tested on the stand-alone mechanism and then again under realistic robot motion, including tests with tilt and uneven terrain. The final geometry is intentionally simple because reliability matters more than a theoretically elegant but fragile sequence.

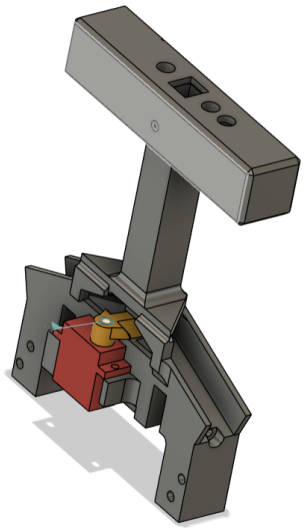


Figure 3: CAD view of the directional rescue-kit mechanism with servo-driven release geometry and vertical storage tower.

3.4 Manufacturing Strategy

All custom mechanical parts were designed by us. Nearly all non-commercial structural parts are 3D printed in PLA-CF, including the chassis, upper plate, axle structures, sensor carriers, bumper bar, rescue mechanism parts, and wheel rims. The wheels are completed by casting silicone directly onto the printed rims using a dedicated mold process. Threaded inserts are installed by heat, and rotating joints use ball bearings for repeatable alignment and lower friction.

This manufacturing strategy fits our priorities well. Additive manufacturing made fast iteration possible, while the cast tires delivered a contact surface that would have been difficult to achieve with printed material alone. At the same time, modular printed parts make maintenance straightforward because damaged components can be reprinted and replaced individually without a major rebuild.

3.5 Mechanical Reliability and Iteration

Mechanical testing focused on full-maze traversal rather than isolated bench measurements alone. We used a large maze with ramps, stairs, bumpers, and even some obstacles that were harsher than typical competition elements. This gave early feedback about where the drivetrain, suspension, and dispenser still needed work. The most relevant observations were whether the robot kept all wheels engaged, whether it maintained heading through repeated obstacle transitions, and whether the rescue kits still deployed correctly under tilt.

The CAD views in Figure 1 also reflect a second mechanical priority: every important subassembly should remain visible and individually replaceable. This is especially relevant for a competition robot that is repeatedly transported, tested, and repaired under time pressure.

One of the strongest mechanical design principles was serviceability. The robot is intentionally modular: the front axle, rear axle, suspension linkage, dispenser, top sensor plate, and bumper bar can be understood as separate subassemblies. This improves repair speed during the season and also supports documentation, because each important subsystem can later be shown with its own CAD view or photograph.

4 Electronic Design and Embedded Integration

The electronic system was redesigned to turn a previously cable-heavy robot into a cleaner, centrally integrated platform.

4.1 Electronics System Overview

The final architecture is organized around a main Raspberry Pi, a dedicated microcontroller layer, and a custom central PCB that distributes both power and signal connectivity. The Raspberry Pi runs high-level functions such as AI inference, mapping, navigation, and the operator dashboard. The microcontroller is used for sensor acquisition and hardware-near tasks, which keeps time-critical I/O separate from Linux-side image processing and path planning. All major sensors are connected through the main board, which also serves as the physical backbone of the wiring concept.

This structure was one of the most important changes compared to earlier seasons. Instead of allowing each subsystem to bring its own cabling solution, we standardized the interfaces and routed them through one board. That reduced wiring complexity, made modules easier to replace, and improved assembly quality.

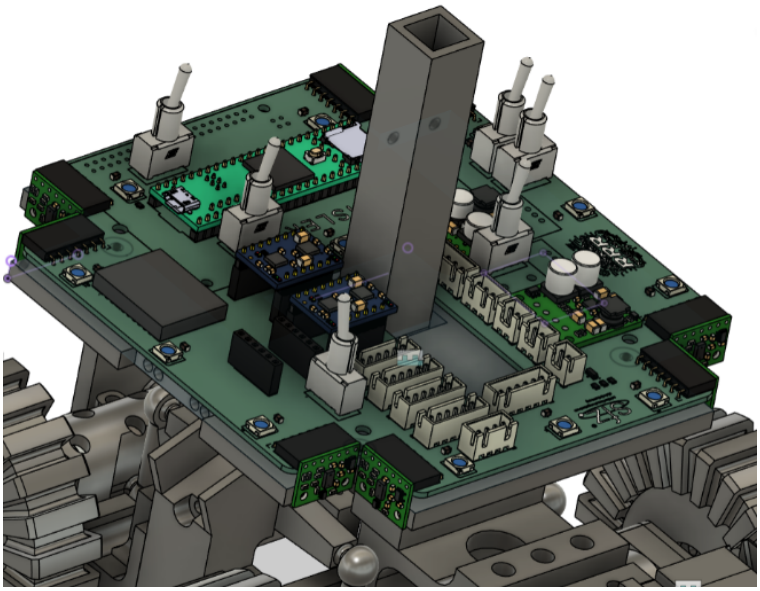


Figure 4: Integrated upper electronics deck with the custom main PCB, modular sensor connectors, compute mounting, and central mechanical tower interface.

4.2 Power Distribution and Protection

Power is supplied by a 3S LiPo battery connected to the main PCB. From there, four regulator stages generate the voltages required by the system: 12 V at 2.5 A for the motors, 5 V at 5 A for the Raspberry Pi, 5 V at 3 A for the microcontroller and NeoPixel-related loads, and 3.3 V at 3 A for the sensors. This separation was chosen so that high-current compute and noisy actuation loads would not directly share the same small-signal rail with the sensor network.

At the current stage, power integrity and EMC have not been the main limiting factor in testing. The more important reliability gain came from physically cleaner routing and modular connectors. Nevertheless, the multi-rail design already makes later debugging easier because individual groups of consumers can be reasoned about separately.

4.3 Sensor Subsystem

The sensor stack reflects the division between navigation, terrain interpretation, and victim handling. Eight VL53L1X time-of-flight sensors measure wall distances around the robot and provide the main local geometry for maze navigation. Three forward-looking multi-target ToF sensors detect objects ahead of the robot and are also used for ramp and obstacle awareness. An APDS9960 RGB sensor detects relevant floor colors such as black and blue. A WT901 gyroscope supplies angular information for precise turns and ramp interpretation. Motor encoders provide motion-related feedback, while a QTRXL-HD-01A reflectance sensor supports silver detection. In addition, tactile switches integrated in the front bumper give direct physical feedback during contact events.

Victim detection is handled by two side-facing cameras mounted at roughly 90 mm height and angled outward by 90 degrees. This placement allows the robot to inspect side walls while continuing to navigate the maze. Mechanically, the cameras are fixed to the upper structure, where they share a stable reference with the main electronics and side sensors.

4.4 Embedded Communication and Motor Control

The Raspberry Pi and the microcontroller communicate over a serial connection. On the embedded side, sensor data is aggregated and forwarded in a normalized form before the Pi uses it for mapping or state decisions. Within the board-level network, the robot uses I2C for several sensors, SPI for the multi-target ToF sensors, and analog acquisition where required by the reflectance sensor.

Motor control is separated from sensor handling. The robot uses four DC drive motors, and the software is structured around front and rear motor pairs. The same layered idea is used for the rescue-kit servo: high- level victim logic decides when and to which side a kit should be dropped, while a lower layer performs the actual actuation sequence. This separation made the software easier to port and reduced tight coupling between navigation and raw hardware access.

Figure 4 also shows why this integration approach mattered mechanically. The custom board did not only reduce wiring complexity; it also defined where sensor modules, regulators, and compute hardware sit relative to each other, which in turn simplified assembly and diagnostics.

4.5 Team-Manufactured Electronics

The custom main PCB is the key team-manufactured electronic artifact of the season. We designed it in KiCad and had it manufactured by Aisler. Instead of splitting the electronics into several small boards, we chose one main board that all sensors connect to through pin sockets or JST-XH connectors. The cables were crimped in house, which gave us control over exact lengths and routing.

This single-board strategy is not spectacular in isolation, but it had a strong practical effect. It removed a major source of wiring confusion, reduced assembly friction, and made the robot easier to understand during debugging. In other words, the electronic innovation is less about an exotic circuit and more about disciplined integration quality.

4.6 Electronics Validation and Fault Handling

Electronics tests concentrated on the basics that most strongly affect integration speed: sensor readout, power distribution, and connection faults. We repeatedly verified whether all sensors were available through the main board and whether failures could be narrowed down to a connector, a cable, or a software-side parsing issue. This was especially important because many software problems only become interpretable once the electrical layer is known to be stable.

An important general outcome was that the electronics no longer dominate the robot's failure profile. The season's more difficult remaining issues are currently in navigation and target classification rather than in board bring-up or wiring chaos. That alone makes the new electronic architecture a meaningful success.

5 Software Architecture

Software is one of the strongest parts of the current system and one of the main reasons the new robot can be tuned quickly.

5.1 Runtime Architecture

The runtime is centered on the Raspberry Pi and is split into clear functional modules. The main program starts hardware polling, optional telemetry, a separate camera-detection thread, and then enters the mapping and navigation loop. Low-level sensor acquisition is abstracted behind a sensor bridge and hardware layer. Motion execution is handled through dedicated movement and motor-control modules. Global navigation state is owned by the map. In parallel, a lightweight local server provides telemetry, camera streams, sounds, and a victim image gallery for debugging.

This modular split was important for development speed. It allowed us to improve mapping, detection, and debugging infrastructure before the final hardware was ready, and it reduced the amount of code that needed to change when the new robot was assembled.

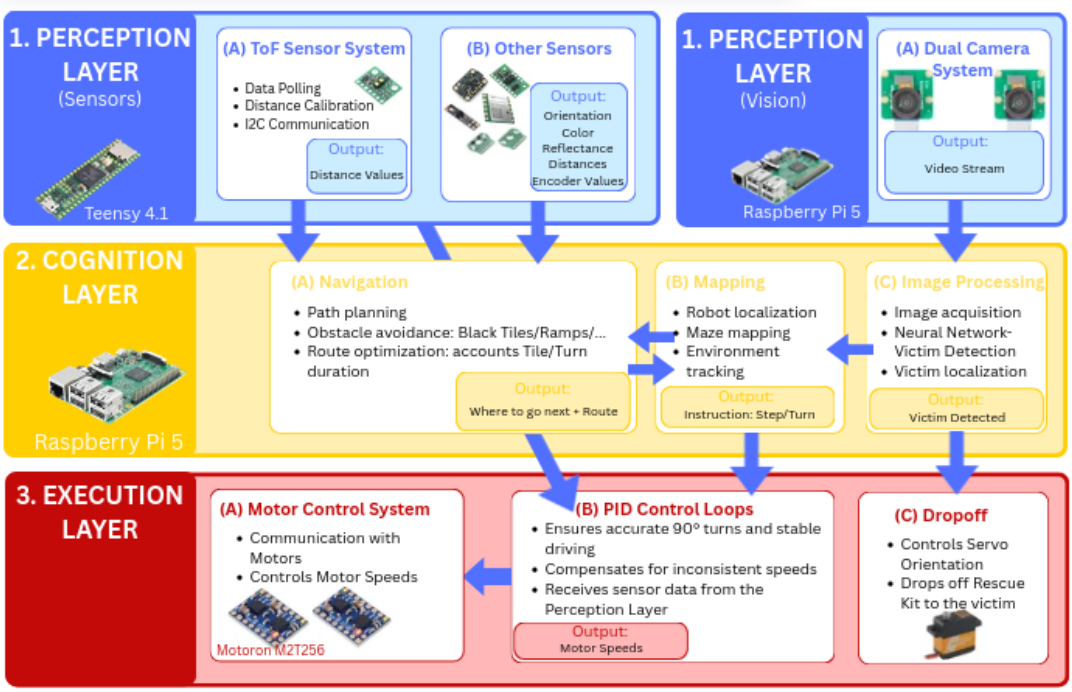


Figure 5: Software and control architecture from perception through cognition to execution. The diagram reflects the runtime split between sensor ingestion, navigation, mapping, image processing, motor control, and dropoff control.

5.2 Sensor Ingestion and Normalization

The embedded side publishes structured distance, color, and gyro values over serial, and the Pi-side bridge parses these frames into indexed sensor arrays. A dedicated hardware layer then turns them into robot-level queries such as side distances, gyro axes, floor colors, button states, and servo commands. This step is more important than it looks, because mapping and movement should not care about serial syntax or bus-specific details. They need stable semantic inputs.

Color and silver detection are additionally filtered in software. We experimented with using a direct silver sensor path, but the more important competition advantage came from the checkpoint logic that could recover map state even when floor-based silver recognition was not sufficiently reliable in practice.

5.3 Mapping, Navigation, and Checkpoint Recovery

Internally, the robot maintains a three-dimensional tile map that stores visited state, wall information, tile color, ramp semantics, per-tile durations, and already served victims. Direction choice is based on a shortest- path style search that prefers unexplored areas first and later computes a return path to the start tile for exit-bonus behavior. Ramp transitions are treated explicitly in the map, including cross-layer edges that connect the corresponding up and down entries.

The most important recovery feature is the persistent silver checkpoint system. The current software saves map snapshots together with a compact "fingerprint" of local geometry derived from front, left, right, and back distance readings plus gyro heading. When a reset or recovery is required, the robot does not simply reload the latest checkpoint. Instead, it compares the current fingerprint against stored checkpoints and chooses a map state that best matches the physical situation. This continuous fingerprint-based silver checkpoint logic often saved runs that would otherwise have lost most of their map information, and it improved exit-bonus success even when direct silver detection on the floor was unreliable.

This is also an example of our broader software philosophy. Rather than relying on a single perfect sensor event, the robot uses redundant structural information to make recovery decisions. That makes the system more tolerant to local sensing errors and was one of the most valuable practical improvements of the season.

Competition-critical software features. Persistent checkpoint recovery preserves map state after failures, the dual-model victim pipeline separates letters from colored targets, the new victim AIs run on dual ultra-wide-angle camera views, and the local dashboard makes navigation and perception failures inspectable during the same test cycle.

5.4 Victim Detection and Delivery Logic

Victim handling uses two side-mounted ultra-wide-angle cameras and a dual-model inference pipeline. One model classifies letter victims, while the second handles colored targets. Both models were trained entirely on synthetic data. We invested significant effort into noise modeling and augmentation in order to reduce the simulation-to-reality gap and to avoid the large manual labeling effort that would have been required for a purely real-image dataset.

Detection is not consumed naively. The software normalizes detections into common victim labels, keeps a short memory window over the end of a forward step, and uses voting while the robot pauses and blinks. This allows the robot to recover victims that were seen only briefly near the edge of a step and to refine uncertain classifications before dropping a rescue kit. Target detections are also re-evaluated through a ring-code vote, while letter-like detections are collapsed by majority vote. The robot can store snapshot images of detected victims into a local gallery, which later helps with debugging and model review.

Once a victim has been accepted, the software determines the wall side, optionally corrects alignment based on relative horizontal image position, triggers the directional drop mechanism, and stores the result inside the map so the same wall is not processed repeatedly. This makes victim handling part of the overall navigation state rather than an isolated camera event.

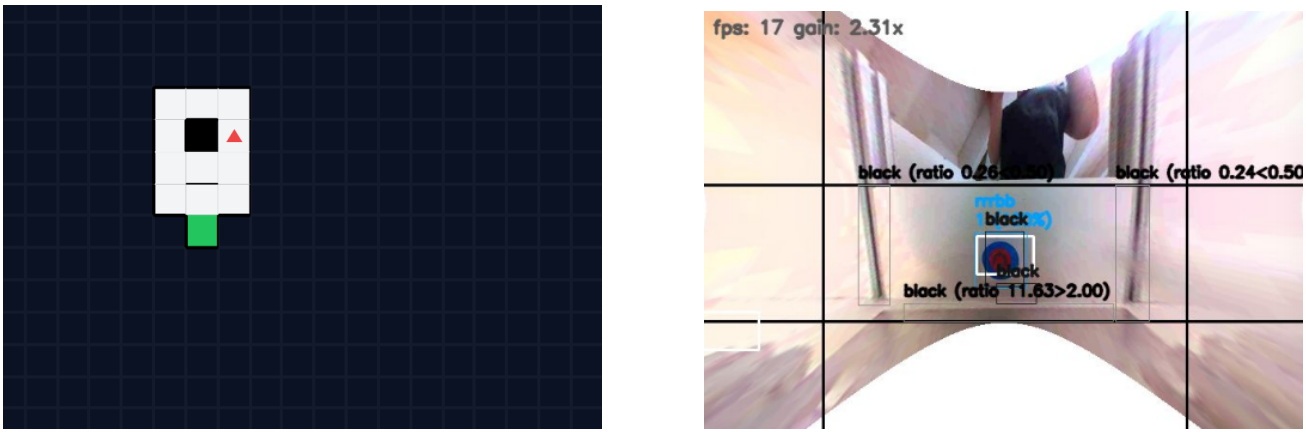


Figure 6: Two typical software views used during development: a live map representation for navigation analysis and a camera debug frame with visual detection overlays.

5.5 Telemetry, Dashboard, and Debugging Infrastructure

The local telemetry dashboard was used extensively during development and is one of the most valuable support tools in the project. It visualizes live sensor values, the robot's current map, camera streams, sound events, and a stored gallery of victim images. A dedicated map view makes it easier to inspect navigation mistakes, and the interface supports camera-mode switching for debugging thresholded or raw views.

This dashboard changed how quickly we could evaluate new ideas. Instead of guessing why a run failed, we could compare live map evolution, visual detections, sensor states, and resulting robot behavior. In practice, that observability was essential for debugging both mapping and AI behavior and should be regarded as part of the engineering system, not just as an operator convenience.

The images in Figure 6 illustrate this workflow directly. The left view condenses the map state into a fast-to-read navigation picture, while the right view exposes what the victim pipeline sees at runtime, including thresholds, bounding boxes, and tentative labels.

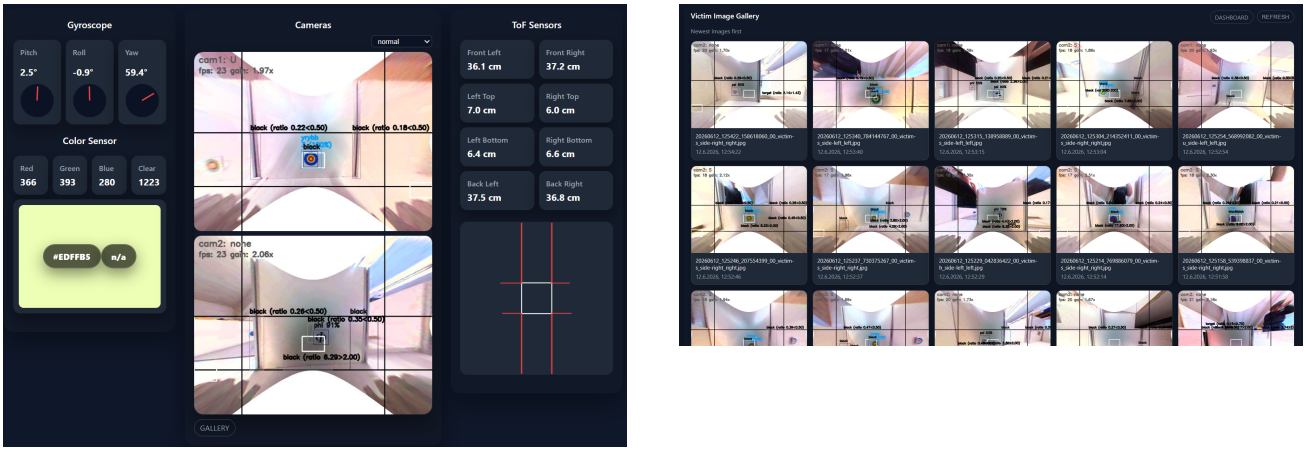


Figure 7: Support tooling used throughout testing. Left: live dashboard with gyro, color, ToF, and camera panels. Right: victim gallery used to review stored detections after runs.

5.6 Software Validation and Regression Strategy

Software testing was driven mainly by repeated maze runs, targeted subsystem isolation, and direct comparison of before-and-after behavior in the dashboard. We regularly ran complete mazes with

victim detection fully enabled, letter-only, target-only, or completely disabled, depending on which failure needed to be isolated. Run duration from start to exit bonus was tracked as a practical high-level performance signal.

The largest remaining regressions are still related to navigation and target ambiguity. Mapping can still fail when the robot drives into a tile that is effectively closed, and some colored target classifications remain sensitive, especially for visually ambiguous yellow or green cases. The response to these failures was not only to add more logic but often to remove or simplify logic that was not earning its place. This reduction-focused strategy improved consistency and kept the software aligned with real competition performance instead of feature count.

6 Performance Evaluation

Evaluation focused on repeated practical runs, not isolated best-case demonstrations.

6.1 Test Methodology

Testing was carried out on several levels. Mechanical and dispenser subsystems were first checked individually. Sensor readout and power integrity were validated during board bring-up. After that, we focused mainly on integration tests and repeated full-maze runs, because many important failures only appear once mechanics, embedded sensing, high-level mapping, and victim handling interact under time pressure.

The most representative test environment was a large practice maze with ramps, stairs, bumpers, and obstacles. Some of these obstacles were intentionally harsher than what is normally expected in competition, which made them useful as stress tests. For software-specific debugging, the same maze could be driven with different feature combinations enabled or disabled, making it possible to isolate whether a problem came from movement, mapping, victim recognition, or the interaction between them.

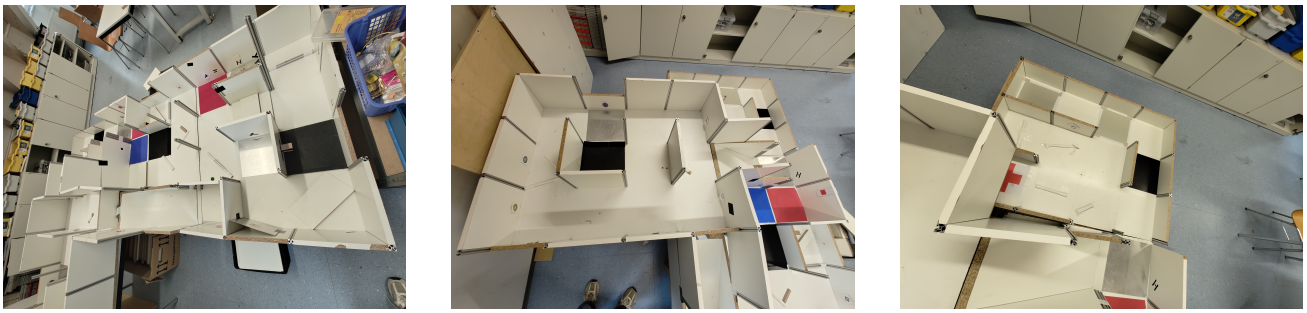


Figure 8: Representative large-scale maze setups used for repeated full-run and integration testing.

6.2 Reliability Metrics

The current architecture was tested much more extensively in software than in the final fresh hardware build, because the robot itself was completed relatively late. Nevertheless, the software core had already gone through more than one hundred runs on earlier hardware with closely related logic, and this significantly reduced transfer risk during the final weeks.

Metric	Observed value	Interpretation
Victim detection accuracy	about 95%	The AI pipeline is already competition-relevant and one of the robot's strongest features
Successful kit drops	about 98%	The mechanical and software handoff for victim response is highly reliable once a victim is accepted
Successful silver recoveries	about 70%	Fingerprint-based checkpoint recovery is valuable, especially when direct silver sensing is unreliable
Full run completion rate	about 50%	The robot is capable of strong runs but still limited by navigation edge cases and target-related errors
Exit bonus success	about 40%	Return behavior works often enough to matter, but still depends on map health and late-run consistency
Average noticeable errors per run	about 5	The failure count is still too high for top-end consistency and defines the current tuning focus

Current limits. The strongest remaining issues are navigation edge cases in effectively closed tiles, ambiguous colored targets, and the still too low rate of fully consistent end-to-end runs on the final hardware.

6.3 Failure Analysis and Corrective Actions

Three examples illustrate how we used these tests. First, stepless driving looked promising in theory but was judged too unreliable in practice, so it was reduced in importance instead of being forced into the final competition stack. Second, when speed-oriented features did not improve consistency, they were removed or de-emphasized. Third, checkpoint recovery was strengthened because analysis showed that preserving map state after a fault was often worth more than trying to prevent every possible fault in advance.

The remaining weaknesses are also well understood. The most persistent software issue is a class of mapping bugs where the robot commits to a motion although the path is effectively blocked. On the perception side, some targets remain difficult when color separation is weak, especially for yellow and green-like appearances. These are exactly the kinds of failures that benefit from the existing dashboard, logs, run videos, and victim snapshot gallery, because the team can compare what the robot "believed" with what actually happened.

6.4 Competition Readiness Assessment

Overall, the robot is already strong in the areas that we intentionally prioritized this season: compact mechanical packaging, good traction and terrain handling, highly usable debugging support, synthetic-data victim recognition, and robust checkpoint-based recovery. The system is not yet perfect, but it has moved well beyond the late-season fragility of earlier years. The main next step is not to add major new features but to convert the current half-reliable full runs into consistently complete runs by reducing navigation edge cases and the remaining false target classifications.

The practice environments shown in Figure 8 matter for this assessment because they exposed the robot to repeated integration stress instead of one-off demonstration runs. That makes the current reliability estimate more useful than a single successful showcase run would be.

7 Conclusion

The 2026 robot is the result of a deliberate shift from rushed iteration toward cleaner system engineering.

Our current Rescue Maze robot is a fully new platform that combines a compact mechanical layout, a much cleaner electronic backbone, and a mature software stack that already contributes strong competitive behavior. The most important design decisions of the season were the central PCB, the coupled passive suspension, the custom wheel concept, and the software emphasis on observability and checkpoint recovery.

In practice, the robot's strongest advantages are not isolated benchmark numbers but the way its subsystems support each other. Good traction helps mapping, the dashboard accelerates debugging, and the continuous fingerprint-based silver checkpoint logic limits the damage of navigation faults. The victim pipeline is also a clear strength, especially because it was trained entirely on synthetic data and still performs well in real tests.

The remaining work is equally clear. We now need to reduce the last mapping edge cases and the remaining ambiguous target classifications so that strong partial runs turn into consistently complete runs. Even with that unfinished work, the robot already represents a substantial engineering step forward from the previous season and provides a solid base for further refinement.